

Database Systems

Assignment#04



23K-2001

BCS-5J

Q1:

a. ACID property/properties violated:

The violation involves
Atomicity and Durability.

b. Reasoning:

Atomicity was violated because the steps of the transaction (driver acceptance, wallet deduction, confirmation update) did not execute as a single all-or-nothing unit.

Some operations were committed (wallet deduction, ride acceptance) while the final confirmation was not completed due to the server crash.

Durability was violated because the system failed to guarantee that once the transaction completed, its effects would survive a crash.

The wallet deduction and driver acceptance were not safely logged/committed before the crash, leading to an inconsistent state visible to the user.

c. Corrected scenario:

A properly designed ACID-compliant process would be:

- Begin a transaction.
- Record driver acceptance in a pending state.
- Reserve (not deduct) the customer wallet amount.
- Write all changes to stable storage using write-ahead logging.
- Commit the transaction only after both updates succeed.
- After commit, show confirmation to the customer.
- If any step fails or a crash occurs before commit, rollback everything so wallet balance and driver acceptance return to their previous state.

This ensures atomicity, consistency, isolation, and durability.

d. Long-term business consequences of frequent violations:

- Loss of customer trust because money is deducted without confirmation.
- Increased refund requests and operational costs.
- Drivers receiving ghost bookings, reducing platform reliability.
- Negative ratings, bad reviews, and poor brand reputation.
- Potential regulatory issues for mishandling digital payments.
- Customers shifting to competitors, resulting in long-term revenue loss.

Q2:

a) Non-serial schedule (interleaved) — one-line timeline

Using notation $r1 = T1.read$, $w1 = T1.write$, $r2 = T2.read$, $w2 = T2.write$.

Schedule S:

1. $r1(P1)$ // T1 reads Q0
2. $r2(P1)$ // T2 reads Q0
3. $w1(P1)$ // T1 writes Q0 - sold_units
4. $w2(P1)$ // T2 writes Q0 + returned_units

Numeric example: Q0 = 100, sold_units = 10, returned_units = 5

$r1$ reads 100, $r2$ reads 100, $w1$ writes 90, $w2$ writes 105 → final = 105 (inconsistent; lost update)

b) Precedence (conflict) graph construction — step-by-step

Conflicts exist when operations by different transactions on same data item and at least one is a write.

Operations in S:

- $r1(P1)$ (T1)
- $r2(P1)$ (T2)

- $w_1(P_1)$ (T1)
- $w_2(P_1)$ (T2)

Conflicting pairs and induced edges:

1. $r_2(P_1)$ before $w_1(P_1)$: Read-Write conflict between T2 and T1 $\rightarrow$ edge $T_2 \rightarrow T_1$.
Reason: T2 read P1 then T1 wrote P1; order matters for serializability.
2. $w_1(P_1)$ before $w_2(P_1)$: Write-Write conflict between T1 and T2 $\rightarrow$ edge $T_1 \rightarrow T_2$.

Graph edges: $T_2 \rightarrow T_1$ and $T_1 \rightarrow T_2 \rightarrow$ **this creates a cycle** ($T_1 \leftrightarrow T_2$).

c) Is the schedule conflict-serializable?

No. Because the precedence graph contains a cycle ($T_1 \rightarrow T_2 \rightarrow T_1$), the schedule is **not conflict-serializable**. It cannot be transformed into a serial execution by swapping non-conflicting adjacent operations.

d) Corrected (serializable) schedule ensuring consistency

Serialize transactions (choose T1 then T2, or T2 then T1). Example: T1 followed by T2. Correct serial schedule (T1 then T2):

1. $r_1(P_1)$ // T1 reads Q0
2. $w_1(P_1)$ // T1 writes Q0 - sold_units
3. commit T1
4. $r_2(P_1)$ // T2 reads updated value (Q0 - sold_units)
5. $w_2(P_1)$ // T2 writes (Q0 - sold_units) + returned_units
6. commit T2

Numeric example (Q0=100): T1 writes 90, then T2 reads 90 and writes 95 $\rightarrow$ final = 95 (correct).

Alternatively, enforce concurrency control:

- Use strict two-phase locking (S2PL): acquire exclusive lock on P1 for the transaction that writes, preventing the interleaving that produced the lost update.

- Or use optimistic concurrency with version checks (compare-and-swap / detect write-write conflict and retry).

e) Real-world loss from such anomalies

- **Inventory mismatches:** out-of-stock or overstock decisions based on incorrect data → lost sales or excess holding cost.
- **Customer service failures:** selling items that aren't actually available (backorders), leading to refunds and reputation loss.
- **Financial misstatement:** revenue and cost numbers become wrong, affecting accounting and forecasting.
- **Operational inefficiency:** time spent in reconciliation, manual fixes, and emergency reorders.
- **Supply chain disruption:** wrong reorder quantities affecting suppliers and logistics.
- **Potential legal/contractual penalties** if SLAs or vendor/service obligations are violated.

Q3:

Schedule S1

1. Precedence Graph for S1

We check conflicts (RW, WR, WW) on the same data item.

Conflicts identified:

- **On a:**
 $r1(a) \rightarrow r4(a) \rightarrow w1(a)$
 Edges: $T4 \rightarrow T1$

- **On b:**
 $r2(b) \rightarrow r1(b)$ (no edge; both reads)
 $r1(b) \rightarrow w2(b) \rightarrow \text{conflict} \rightarrow \text{edge } T1 \rightarrow T2$
- **On c:**
 $r3(c) \rightarrow r2(c)$ (no edge; both reads)
 $r2(c) \rightarrow w3(c) \rightarrow \text{conflict} \rightarrow \text{edge } T2 \rightarrow T3$
- **On d:**
 $r4(d) \rightarrow r3(d)$ (read-read, no edge)
 $r3(d) \rightarrow w4(d) \rightarrow \text{conflict} \rightarrow \text{edge } T3 \rightarrow T4$

Final edges in S1 graph:

$T1 \rightarrow T2$

$T2 \rightarrow T3$

$T3 \rightarrow T4$

$T4 \rightarrow T1$

Precedence Graph Result:

This forms a **cycle**:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T1$

2. Is S1 serializable?

No, S1 is NOT conflict-serializable due to the cycle.

3. Equivalent serial schedule:

None (not serializable).

Schedule S2

1. Precedence Graph for S2

Conflicts:

- **On d:** $r_4(d) \rightarrow r_3(d)$ (RR, no edge)
 $w_4(d) \rightarrow r_3(d) \rightarrow \text{conflict} \rightarrow T_4 \rightarrow T_3$
- **On c:** $r_3(c) \rightarrow r_2(c)$ (RR, no edge)
 $w_3(c) \rightarrow r_2(c) \rightarrow \text{conflict} \rightarrow T_3 \rightarrow T_2$
- **On b:** $r_2(b) \rightarrow r_1(b)$ (RR, no edge)
 $w_2(b) \rightarrow r_1(b) \rightarrow \text{conflict} \rightarrow T_2 \rightarrow T_1$
- **On a:** $r_4(a) \rightarrow r_1(a)$ (RR, no edge)
 $w_1(a)$ does not conflict with T_4 's earlier writes (different order), so **no extra edges**.

Final edges in S2 graph:

$T_4 \rightarrow T_3$

$T_3 \rightarrow T_2$

$T_2 \rightarrow T_1$

Graph shape:

A straight chain:

$T_4 \rightarrow T_3 \rightarrow T_2 \rightarrow T_1$

2. Is S2 serializable?

YES, S2 is conflict-serializable.

(No cycles in the graph.)

3. Equivalent serial schedule:

The only serial order consistent with the graph is:

T4 → T3 → T2 → T1

(Unique → because it forms a total order.)

Q4:

1. The concurrency problem:

The schedule contains the “lost update” problem.

Both transactions read the same initial value (2000) before either writes back. Each computes its own update independently. When T1 writes 1700, and later T2 writes 2800, the value written by T1 is completely overwritten and lost.

2. Total_Amount:

The final value written is 2800.

3. Correct Total_Amount if executed serially:

a. T1 → T2:

Start = 2000

After T1 applies discount: $2000 - 300 = 1700$

After T2 adds product: $1700 + 800 = 2500$

Correct total = 2500

b. T2 → T1:

Start = 2000

After T2 adds product: $2000 + 800 = 2800$

After T1 applies discount: $2800 - 300 = 2500$

Correct total = 2500

In both serial orders the consistent final amount is 2500.

Q5:

(a) $r1(X); w1(X); r3(X); r2(X); w3(X)$

Conflicts:

- $w1(\text{pos2})$ before $r3(\text{pos3}) \rightarrow T1 \rightarrow T3$
- $w1(\text{pos2})$ before $r2(\text{pos4}) \rightarrow T1 \rightarrow T2$
- $r2(\text{pos4})$ before $w3(\text{pos5}) \rightarrow T2 \rightarrow T3$

Graph edges: $T1 \rightarrow T2 \rightarrow T3$ (and $T1 \rightarrow T3$). Acyclic.

Serializable: **Yes**.

Equivalent serial schedule(s): one valid serial order is **T1, T2, T3** (this respects all edges).

(b) $r1(X); r3(X); w3(X); w1(X); r2(X)$

Conflicts:

- $r1(\text{pos1})$ before $w3(\text{pos3}) \rightarrow T1 \rightarrow T3$
- $w3(\text{pos3})$ before $w1(\text{pos4}) \rightarrow T3 \rightarrow T1$

Graph edges: $T1 \rightarrow T3$ and $T3 \rightarrow T1 \Rightarrow$ cycle.

Serializable: **No** (cycle between T1 and T3). Not conflict-serializable.

(c) $r3(X); r2(X); w3(X); r1(X); w1(X)$

Conflicts:

- $r2(\text{pos2})$ before $w3(\text{pos3}) \rightarrow T2 \rightarrow T3$
- $w3(\text{pos3})$ before $r1(\text{pos4}) \rightarrow T3 \rightarrow T1$
- $r2(\text{pos2})$ before $w1(\text{pos5}) \rightarrow T2 \rightarrow T1$ (read by T2 then write by T1)

Graph edges: $T2 \rightarrow T3 \rightarrow T1$ (and $T2 \rightarrow T1$). Acyclic.

Serializable: **Yes**.

Equivalent serial schedule(s): a valid serial order is **T2, T3, T1**.

(d) $r3(X); r2(X); r1(X); w3(X); w1(X)$

Conflicts:

- $r2$ (pos2) before $w3$ (pos4) $\rightarrow T2 \rightarrow T3$
- $r1$ (pos3) before $w3$ (pos4) $\rightarrow T1 \rightarrow T3$
- $w3$ (pos4) before $w1$ (pos5) $\rightarrow T3 \rightarrow T1$

Graph edges include $T1 \rightarrow T3$ and $T3 \rightarrow T1 \Rightarrow$ cycle.

Serializable: **No** (cycle between T1 and T3). Not conflict-serializable.